AI speed. Human accountability.

CREDSHIELDS · SMART CONTRACT AUDIT

Security Assessment

Sivo

Confidential — Prepared for the Sivo Engineering Team

This report presents the findings of a smart contract security audit conducted against Sivo's Uniswap Hook Contracts. 5 out of 7 findings are fixed. Findings were identified through manual review of the contract source by the CredShields' security team.

0	0	2	3	2	7
CRITICAL	HIGH	MEDIUM	LOW	INFO	TOTAL FINDINGS

CredShields Technologies Pte. Ltd. · Singapore
credshields.com · info@credshields.com

CONFIDENTIAL

/ CONTENTS

Table of Contents

01 Executive Summary

02 Vulnerability Summary

03 Detailed Findings

M001 Single NAV feed prices USDC and USDT at par, enabling cross-asset depeg extraction MEDIUM

M002 Partial `cancelWithdraw` drives a live request below `minRequestShares`, defeating the dust guard MEDIUM

L001 Permissionless `fill` lets a third party choose a redeemer's settlement price LOW

L002 Cancelled-request tombstones starve the bounded auto-fill pass LOW

L003 Unbounded `requestIdsOf` growth bricks a controller's own `cancelWithdraw` LOW

I001 Unbounded mint at the last-published NAV dilutes existing SLP holders INFO

I002 Centralization risk: privileged roles concentrate pricing, custody, and code control INFO

– Lead Reviewers & Signatories

/ 01: THE PICTURE AT A GLANCE

Executive Summary

A smart contract security audit was conducted for Sivo, identifying a total of **7 findings**. These findings were categorized by severity as 0 Critical, 0 High, 2 Medium, 3 Low, and 2 Informational.

While no Critical or High severity issues were identified, two Medium severity findings warrant attention. These include a single NAV feed for USDC and USDT priced at par, which could enable cross-asset depeg extraction, and a partial `cancelWithdraw` function that can drive a live request below `minRequestShares`, defeating its dust guard.

Other findings encompassed three Low severity issues, such as permissionless fill allowing a third party to choose a redeemer's settlement price, and two Informational items. Following the initial assessment, a retest confirmed that **5 out of the 7 identified findings have been successfully fixed**.

Audited scope

1. Proxy: https://etherscan.io/address/0x0a71509dff50940945538b6c9137d26b1d4468a8
2. Implementation: https://etherscan.io/address/0xbFc609FF828410069F9b21D39a4484e5462A75b1

Retested scope

Deployed Mainnet Contracts:
1. Proxy: https://etherscan.io/address/0x0A71509dfF50940945538B6c9137d26B1D4468A8
2. Implementation: https://etherscan.io/address/0x3aE3Db464C6904582a090e4beD464415B255Ce95

Assessment overview

This engagement involved a comprehensive security audit of Sivo's smart contracts, focusing on identifying potential vulnerabilities, business logic flaws, and adherence to security best practices. Our review process included a detailed manual code review, analysis of the system's architecture and economic model, and evaluation against known attack vectors and common smart contract weaknesses. The objective was to provide Sivo with actionable insights to enhance the security posture and resilience of their protocol.

In total, seven findings were identified during the audit, categorized across various severity levels. We are pleased to report that five of these findings have been successfully fixed upon retest, demonstrating Sivo's commitment to security. The remaining two findings are still open for remediation.

The most significant issues identified were classified as Medium severity, primarily concerning business logic and accounting consistency. Finding **M001: Single NAV feed prices USDC and USDT at par, enabling cross-asset depeg extraction** highlighted a potential vulnerability where an attacker could exploit differences in asset pricing during a depeg event. Additionally, **M002: Partial cancelWithdraw drives a live request below minRequestShares, defeating the dust guard** pointed to an inconsistent accounting flaw that could undermine safeguards designed to prevent dust attacks or manipulate withdrawal requests.

Several Low severity findings were also uncovered, largely related to design choices that could lead to unexpected behavior or minor manipulation. For instance, **L001: Permissionless fill lets a third party choose a redeemer's settlement price** indicated a scenario where external parties could influence transaction outcomes. Another concern, **L003: Unbounded requestIdsOf growth bricks a controller's own cancelWithdraw**, highlighted a potential denial-of-service vector related to an unmanaged array growth. Relatedly, **L002: Cancelled-request tombstones starve the bounded auto-fill pass** presented a scenario where system efficiency could be degraded by unmanaged state.

Finally, two Informational findings provided valuable insights into potential risks and design considerations. **I001: Unbounded mint at the last-published NAV dilutes existing SLP holders** described a front-running risk where new mints could unfairly dilute existing liquidity providers. **I002: Centralization risk: privileged roles concentrate pricing, custody, and code control** outlined the inherent centralization risks associated with privileged administrative roles, emphasizing the importance of robust access controls and operational security for key management.

ENGAGEMENT DETAILS

CLIENT	Sivo
ASSESSMENT TYPE	Smart contract security audit
AUDIT	Sivo
REPORT DATE	25 Aug 2026
START DATE	18 Aug 2026
END DATE	21 Aug 2026
RETEST DATE	24 Aug 2026
TOTAL FINDINGS	7
REVIEWER	Shashank (shashank@credshields.com) · CredShields

/ FINDING SEVERITY DISTRIBUTION

0	0	2	3	2
CRITICAL	HIGH	MEDIUM	LOW	INFO



/ 02: AT A GLANCE

Vulnerability Summary

Every finding from this assessment, ordered by severity. Follow an ID for the full write-up.

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
M001	Single NAV feed prices USDC and USDT at par, enabling cross-asset depeg extraction	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M002	Partial `cancelWithdraw` drives a live request below `minRequestShares`, defeating the dust guard	MEDIUM	Inconsistent Accounting (SCWE-010)	✓ FIXED
L001	Permissionless `fill` lets a third party choose a redeemer's settlement price	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L002	Cancelled-request tombstones starve the bounded auto-fill pass	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L003	Unbounded `requestIdsOf` growth bricks a controller's own `cancelWithdraw`	LOW	Denial of Service (SCWE-087)	✓ FIXED
I001	Unbounded mint at the last-published NAV dilutes existing SLP holders	INFO	Front-Running (SCWE-037)	ACCEPTED RISK
I002	Centralization risk: privileged roles concentrate pricing, custody, and code control	INFO	Single Point of Failure in Administrative Key Management (SCWE-155)	ACCEPTED RISK

/ 03: DETAILED FINDINGS

7 findings across the assessment.

Findings are ordered by severity.

M001 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

FIXED

Single NAV feed prices USDC and USDT at par, enabling cross-asset depeg extraction

DESCRIPTION

The hook is designed so that one SLP token trades against two interchangeable dollar stablecoins, with a single Chainlink SLP/USD NAV feed supplying the only price the system consults. `deposit` and `beforeSwap` mint SLP at that NAV; `requestWithdraw` queues SLP against whichever stablecoin queue the caller names, and `_fill` pays that queue out at the same NAV.
`_quote` and `_fill` both build the asset side of the conversion as `assetBasis = price * 10 ** assetDecimals(asset)`, where `price` is the SLP/USD answer. The stablecoin's own USD price never enters the arithmetic, so one unit of USDC and one unit of USDT are assumed to be worth exactly the same dollar amount at all times. Because SLP is fungible and `requestWithdraw(address asset, uint256 shares)` takes the payout stablecoin as a free parameter with no link to the asset that minted the SLP, the two reserves are joined by a hardcoded 1:1 exchange rate. The invariant that a share is redeemable for its NAV in real dollars is broken the moment either stablecoin trades away from a dollar.
The bug applies whenever the two supported stablecoins diverge in market price, which includes the March 2023 USDC event and the recurring USDT discounts of 2018 and 2022, and it applies to both legs independently: the mint leg over-issues SLP against real backing and needs no counterparty, while the redemption leg drains the stronger reserve and additionally requires free balance in that queue and a position reachable in its FIFO order. It does not apply while both stablecoins hold parity, and the operator can blunt it by pausing, though the pause is reactive and the divergence window is measured in hours.

AFFECTED CODE

- https://etherscan.io/address/0xbFc609FF828410069F9b21D39a4484e5462A75b1#code#F55#L349

BUSINESS IMPACT

An attacker who buys the discounted stablecoin at 0.95 deposits it at par, mints SLP as though it were worth 1.00, and either holds the over-issued SLP or queues it against the healthy stablecoin and claims at par, capturing the full 5 percent spread on every dollar routed through. Deposit size is uncapped (`maxDeposit` returns `type(uint256).max`) and `deposit` bypasses the PoolManager entirely, so the mint leg is limited only by the attacker's capital; the redemption leg is limited by the healthy queue's free balance, which is the entire unswept reserve for that stablecoin. Every existing SLP holder is diluted in proportion, and the loss is irreversible once the SLP is minted, since nothing in the system can claw back or reprice already-issued shares.

RECOMMENDED NEXT STEP

Price each stablecoin against its own USD feed inside `_quote` and `_fill` rather than assuming one token unit equals one dollar, and reject deposits and fills while a stablecoin's own feed sits outside a tight peg band. Where a second feed per asset is not acceptable, bind redemption to the asset that minted the shares by deploying a separate hook and reserve per stablecoin, so a divergence in one can never reach the other's balance.

RETEST

This issue has been fixed as _requireAssetPegged gates deposit/swap/fill/replenish on each stablecoin's own USD feed and peg band, a depegged coin can no longer mint or drain at par.

M002 · INCONSISTENT ACCOUNTING (SCWE-010)

MEDIUM

Partial `cancelWithdraw` drives a live request below `minRequestShares`, defeating the dust guard

✓ FIXED

DESCRIPTION

The withdrawal queue defends itself against dust with a minimum request size. `requestWithdraw` rejects any `shares < minRequestShares()`, reverting with `UniswapHookRequestTooSmall`, and `setMinRequestShares` documents the bound in NatSpec as the guard "against dust-request griefing." The design intent is that every live entry in the FIFO carries at least the minimum, so the number of entries `_fill` must visit stays proportional to genuine withdrawal demand rather than to how many entries an adversary chose to create.

The minimum is a request-time check only, and nothing re-imposes it once an entry exists. `_cancelWithdraw` reduces a request in place, `r.slpRemaining = uint96(available - cancelled)`, and never compares the residual against `minRequestShares()`, so a caller can request at or above the minimum and then cancel down to any value they like: a positive amount below the floor, or a fully zeroed tombstone. With the minimum at 100, `requestWithdraw(asset, 150)` clears the gate and `cancelWithdraw(asset, 60)` leaves the same request live at 90. Because the cancellation walk runs newest-first and drains each entry fully before it partially touches one, a caller who interleaves a request with a cancel produces exactly one sub-minimum entry per pair, and because cancellation returns the SLP, one minimum-sized chunk of capital funds an unbounded number of them in sequence. The invariant that a live queue entry is worth at least the minimum does not hold past the moment of creation.

AFFECTED CODE

- https://etherscan.io/address/0xbFc609FF828410069F9b21D39a4484e5462A75b1#code#F55#L396

BUSINESS IMPACT

An attacker recycles a single minimum-sized SLP position through `requestWithdraw` and `cancelWithdraw`, spread across throwaway addresses to sidestep the quadratic self-cost, and pads the head of a queue with arbitrarily many sub-minimum and zeroed entries for the price of gas alone, since each cancellation returns the principal. Redeemers queued behind the padding stop being served by the bounded auto-fill and wait until someone pays to walk the wall, and the operator's `fill` and `replenish` work scales with the manufactured entry count rather than with real withdrawal demand. No principal is lost and the condition is recoverable by any caller, but the dust-griefing defence that `minRequestShares` exists to provide is nullified, which is precisely the property the setter was added to enforce. Affects every queued redeemer for the asset and whoever bears the fill gas.

RECOMMENDED NEXT STEP

Enforce the floor on the residual inside `_cancelWithdraw`: after reducing each touched request, require `r.slpRemaining == 0 || r.slpRemaining >= minRequestShares()`, so a partial cancellation can never leave a live entry below the minimum. Where operational flexibility to cancel an arbitrary amount is required, cap the number of live entries per controller instead, so fragmentation stays bounded regardless of the residual size.

RETEST

This issue has been fixed as residual floor enforced in _cancelWithdraw, so a partial cancel can't manufacture sub-minimum live entries.

L001 · BUSINESS LOGIC (SC02-LOGICERRORS)

LOW

Permissionless `fill` lets a third party choose a redeemer's settlement price

✓ FIXED

DESCRIPTION

Exits are asynchronous by design: `requestWithdraw` deliberately locks no price, and the NatSpec states that pricing happens when the request is filled. `fill(address asset, uint256 maxIterations)` is the keeper entrypoint that performs that pricing, and it is callable by anyone.

The redeemer supplies no floor, no deadline, and no acceptable-price parameter at any point in the lifecycle, and `_fill` prices the burn at whatever `_freshPrice()` returns at the moment of the call. Whoever calls `fill` therefore selects, within the entire `[priceMin, priceMax]` band, which observation becomes the victim's permanent exit price, and `_claimWithdraw` is explicitly not gated on oracle health afterwards because that price is already locked. The property that a redeemer receives NAV at a moment they control, or at minimum at a moment they consented to, does not hold.

The bug applies whenever a queued request sits at the head with free balance available, which is the normal steady state. The band is the only bound, and the contract's own NatSpec describes it as ratcheted around the current NAV operationally rather than kept tight, so the exploitable width is an operations parameter rather than a code constant. It does not apply to requests that are filled by the bounded auto-fill inside a deposit, since the caller there is a genuine depositor rather than a chosen adversary, though an attacker can equally trigger that path by depositing.

AFFECTED CODE

- https://etherscan.io/address/0xbFc609FF828410069F9b21D39a4484e5462A75b1#code#F55#L486

BUSINESS IMPACT

An SLP holder waits for a downward tick within the band, calls `fill` against queued redeemers ahead of them, and permanently locks those redeemers' exits at a price below the one they would have received minutes later. The redeemers lose the difference between the chosen answer and the fair NAV, on the full notional of their queued shares, with no recourse: the claim is already priced and `cancelWithdraw` no longer reaches the filled portion. The attacker's gain arrives indirectly, as burning shares below true NAV raises the NAV per remaining share they hold, which also means the harm scales with the band width rather than with the attacker's capital.

RECOMMENDED NEXT STEP

Accept a minimum assets-per-share and a deadline on `requestWithdraw`, and revert the fill of any request whose bound the current price fails. Where a per-request bound is unacceptable, restrict `fill` to the `restricted` role and rely on the deposit auto-fill path for permissionless progress.

RETEST

This issue has been fixed as fill is now restricted to the operator, a third party can no longer freely select a redeemer's settlement price.

L002 · BUSINESS LOGIC (SC02-LOGICERRORS)

LOW

Cancelled-request tombstones starve the bounded auto-fill pass

✓ FIXED

DESCRIPTION

Every deposit runs a bounded fill pass over the head of the withdrawal queue so that inbound stablecoin reaches waiting redeemers without a keeper. The bound is `AUTO_FILL_MAX = 5`, chosen to keep router gas estimates predictable.
`_cancelWithdraw` zeroes `slpRemaining` in place rather than removing the entry, and `_fillOne` treats a zeroed entry as a tombstone that costs one full loop iteration to skip (`s.head++; return true`). While five or more tombstones sit at the head, every deposit's auto-fill spends its entire budget skipping them and reserves nothing, so the deposited stablecoin stays in free balance and remains sweepable instead of backing a claim. The property that inbound deposits make progress against the queue holds only when the head is not padded with dead entries.
An attacker manufactures tombstones by submitting requests at `minRequestShares` and immediately cancelling them, recovering one hundred percent of the SLP and paying only gas. Spreading the requests across throwaway addresses avoids the quadratic self-cost described in L-02. The effect is bounded and self-healing rather than permanent: `_fill` persists `q.head` even when it breaks early, so each tombstone is skipped exactly once across the lifetime of the queue, and any caller can clear a backlog with one `fill(asset, largeMax)`. Gas cost per skip is lower than the attacker's cost per request-and-cancel pair, so the griefing is gas-negative for the attacker.

AFFECTED CODE

- https://etherscan.io/address/0xbFc609FF828410069F9b21D39a4484e5462A75b1#code#F55#L418

BUSINESS IMPACT

Legitimate redeemers stop being filled automatically by inbound deposits and must wait for someone to pay the one-time cost of walking the tombstone wall, roughly two thousand gas per dead entry, which for a ten thousand entry backlog is around twenty million gas spread across as many transactions as the caller chooses. No funds are lost and no redeemer's claim is reduced; the harm is degraded liveness for queued redeemers plus a gas cost shifted onto whoever clears the backlog. Fully recoverable by any caller at any time.

RECOMMENDED NEXT STEP

Advance `q.head` past a cancelled request inside `_cancelWithdraw` when that request sits at the head, so cancellation cleans up after itself. Where that is not practical, skip tombstones without decrementing the iteration budget so a wall of dead entries cannot consume an entire auto-fill pass.

RETEST

This issue has been fixed.

L003 · DENIAL OF SERVICE (SCWE-087)

LOW

Unbounded `requestIdsOf` growth bricks a controller's own `cancelWithdraw`

✓ FIXED

DESCRIPTION

`cancelWithdraw` returns unfilled queued SLP by walking the controller's own request list from the newest backwards, which preserves the FIFO priority of their older requests and skips entries that have already been filled.
`requestWithdraw` pushes to `q.requestIdsOf[msg.sender]` on every call and nothing ever removes an entry, including entries that have been fully consumed by `_fill` or fully cancelled. The loop in `_cancelWithdraw` starts at `ids.length` on every invocation and traverses the whole array, so its cost grows linearly with the controller's lifetime request count rather than with their open positions. At roughly four to five thousand gas per iteration, two cold storage reads each, a controller past about six thousand historical requests can no longer fit the call in a block.
The bug is reachable only by the affected controller's own activity, since `requestWithdraw` hardcodes `controller = msg.sender` and no operator variant exists to inflate a victim's array. `minRequestShares` bounds how cheaply the array can be grown. It does not lock funds: `_fill` still fills the controller's pending shares and `claimWithdraw` still pays them out, so the loss is the cancellation option rather than the principal, unless the protocol simultaneously stops supplying fill budget.

AFFECTED CODE

- https://etherscan.io/address/0xbFc609FF828410069F9b21D39a4484e5462A75b1#code#F55#L368

BUSINESS IMPACT

A market maker or automated strategy that cycles through the queue accumulates entries until `cancelWithdraw` reverts on out-of-gas permanently, at which point their queued SLP can leave only through the fill path on the protocol's schedule rather than on demand. Affects only that controller, and only their ability to reverse a request; the shares themselves remain redeemable. Recoverable by using a fresh address for future requests, but never recoverable for the affected address, which has no pruning path.

RECOMMENDED NEXT STEP

Swap-and-pop fully consumed request identifiers out of `requestIdsOf` as `_fillOne` and `_cancelWithdraw` retire them. Expose a paginated cancellation that accepts an explicit request identifier list or a start cursor so a controller can always bound the work in one transaction.

RETEST

This issue has been fixed as cancel walk prunes retired ids and drains newest-first.

I001 · FRONT-RUNNING (SCWE-037)

Unbounded mint at the last-published NAV dilutes existing SLP holders

INFO

ACCEPTED RISK

CVSS 0.2

DESCRIPTION

Both mint paths, the router-facing `beforeSwap` and the direct `deposit` entrypoint, price shares off `_freshPrice()`, which returns the most recent Chainlink round that passes the freshness and band checks. The design intent is that a depositor always pays exactly the current net asset value of the shares they receive.

A Chainlink NAV feed publishes discretely, on a heartbeat or when the answer moves past a deviation threshold, while the underlying value it tracks accrues continuously. Between publications `_freshPrice()` returns a price that is correct as of the last round and stale as of now, and neither mint path charges a spread, imposes a size cap, enforces a cooldown between minting and becoming eligible for `requestWithdraw`, or compares the answer against the previous round to detect that an update is imminent. The invariant that a depositor pays present value is therefore violated by exactly the accrued-but-unpublished drift, and an attacker captures it by minting immediately before the update lands.

The bug applies on every publication boundary and needs no special market state, no counterparty, and no privileged role. The magnitude is set by the feed's configuration rather than by anything in this codebase: a tight heartbeat on a slow-yielding fund leaks single-digit basis points per cycle, while a wide deviation threshold or a discretely marked valuation (a coupon landing, a monthly or quarterly revaluation) makes the entire step capturable. The band in `OracleGate` provides no defence, because a stale price is by construction an in-band price. The attacker does not need the queue to have budget, since the mint leg alone realises the gain in the form of SLP they can hold and exit later.

AFFECTED CODE

- https://etherscan.io/address/0xbFc609FF828410069F9b21D39a4484e5462A75b1#code#F55#L306

BUSINESS IMPACT

An attacker monitoring the feed deposits into `deposit` immediately before a NAV update and receives `X / p_stale` shares that are worth `X * p_new / p_stale` the instant the round lands, transferring the difference from every existing SLP holder to themselves at zero risk and with no upper bound on size. The dilution is irreversible: the shares are minted at the wrong price and nothing in the system can reprice or burn them without the holder's cooperation.

RECOMMENDED NEXT STEP

Charge an entry spread that exceeds the feed's maximum inter-round drift, or cap mints per feed round so no single transaction can absorb an entire update. Add a minimum holding period between minting and `requestWithdraw` eligibility so the round trip cannot close inside the arbitrage window, and reject deposits when `latestRoundData` is older than a fraction of the heartbeat rather than only when it exceeds `maxStaleness`.

RETEST

Client Comment:

As the finding notes, the magnitude here is set by the feed's configuration, and we believe ours keeps it negligible. The SLP/USD feed publishes the fund's accounting NAV daily (a 90,000 s staleness bound is enforced on-chain), and the NAV accrues at a predictable yield rate with no discretely marked events. The drift available at a publication boundary is therefore about one day of fund yield — single-digit basis points — and the feed's rate bounds (currently min_rate 1.00 / max_rate 1.02) and ripcord circuit breaker cap any anomalous step.

I002 · SINGLE POINT OF FAILURE IN ADMINISTRATIVE KEY MANAGEMENT (SCWE-155)

INFO

ACCEPTED RISK

Centralization risk: privileged roles concentrate pricing, custody, and code control

DESCRIPTION

The hook is a custodial instrument by design: deposits mint fresh SLP against a stablecoin that becomes immediately sweepable, and exits are honoured from balance the operator controls. A single AccessManager authority, set once in `initialize` and never re-derivable from an unprivileged call, gates every `restricted` function, so the entire trust surface collapses onto whichever addresses hold those roles in the manager.

The concentrated powers are broad and, in several cases, irreversible. `setOracle` retargets the only price the system consults and the band around it, so the authority can point pricing at a feed it controls and then mint or lock redemptions at any value it chooses; this is the sharpest edge, because it is a full economic override reachable in a single call rather than a bounded parameter. `sweep` sends the whole unreserved balance to an arbitrary recipient, `_authorizeUpgrade` is `restricted` with an empty body so the implementation can be replaced wholesale, `pause` freezes user exits while `sweep` stays open. Beneath all of this the hook holds SLP mint authority through `ISLP.mint`; whether Sivo operations hold an independent mint or burn role on the same token is an open item, and if they do the share supply is administratively controllable as well.

AFFECTED CODE

- https://etherscan.io/address/0xbFc609FF828410069F9b21D39a4484e5462A75b1#code#F55#L43

BUSINESS IMPACT

An authority that is compromised, malicious, or merely careless can reprice the protocol, drain the free balance, replace the implementation, or freeze user exits, and can do so against every user of both pools at once; the pricing and upgrade powers are unrecoverable after the fact, since mints clear permanently and an installed implementation controls its own storage. Even under an honest operator, the immediate sweepability of deposits means depositors carry ongoing solvency risk that the contract does not bound: if swept stablecoin is not replenished, queued redemptions cannot be filled regardless of the honesty of the accounting.

RECOMMENDED NEXT STEP

Document the custodial trust model explicitly for depositors, including that deposits are immediately sweepable and that redemption solvency rests on operator replenishment rather than on an on-chain reserve.

RETEST

The trust model is documented.

DISCLOSURE

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or issue and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

REVIEWER(S) & CONTACT

Shreyas Koli
Security Auditor II

Prasad Kuri
Security Auditor

Neel Shah
Security Auditor

LEAD REVIEWERS & REPORT SIGNATORIES



Shashank
Co-founder & CEO · CredShields

Security analyst and penetration tester with **12 years of experience** in pentesting and **5 years** in managing bug bounty programs. Listed in **Google, Facebook, Apple,** and 30+ websites' hall of fame for reporting valid security issues.

CVE-2017-7063 CVE-2017-7062 CVE-2017-2458



Aditya Dixit
Audit & Research Lead · CredShields

Senior security researcher and penetration tester with **10+ years** in cybersecurity. Led pentest teams at **Cobalt and HackerOne**; previously at Binary.com. Executed **500+ pentests** across banking, fintech, payments, healthcare, telecom, and media, covering web, mobile (iOS/Android), network, cloud, code review, and smart contracts.

OSCP CREST CPSA AWS Security Specialty AWS Cloud Practitioner Qualys VMDR